

Customer Churn Prediction & LTV Engine

Week 3 • Day 2

LTV Regression Modelling

Build a regression model to predict future customer revenue

■ Day Objectives

- ◆ Understand regression vs classification
- ◆ Train a Gradient Boosting Regressor to predict `predictive_ltv`
- ◆ Evaluate with MAE, RMSE, and R^2 metrics
- ◆ Visualise predicted vs actual LTV
- ◆ Save the LTV regression model for API use

■ 1. Regression for LTV Prediction

While our churn model is a **classifier** (predicts Yes/No), the LTV model is a **regressor** — it predicts a continuous numeric value (expected revenue in dollars). We use the same engineered features but a different target: `predictive_ltv` instead of churn.

Regression Evaluation Metrics

- **MAE (Mean Absolute Error):** Average absolute dollar error. Easy to interpret: '\$45 average error'
- **RMSE (Root Mean Squared Error):** Punishes large errors more. More sensitive to outliers.
- **R^2 (R-squared):** Proportion of variance explained. 1.0 = perfect, 0 = no better than predicting the mean.

11

■ src/models/ltv_regression.py

```
import pandas as pd
import numpy as np
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import matplotlib.pyplot as plt
import joblib
```

```
df = pd.read_csv('data/processed/telco_ltv.csv')
```

[illegible]

```
drop_cols = ['customerid', 'churn', 'loyalty_segment', 'ltv_segment',
             'historical_ltv', 'total_ltv', 'predictive_ltv',
             'churn_probability', 'expected_remaining_months']
```

```
drop_cols = [c for c in drop_cols if c in df.columns]
```

```
X = df.drop(columns=drop_cols)
```

```
y = df['predictive_ltv']
```

```
# One-hot encode categoricals
```

```
cat_cols = X.select_dtypes('object').columns.tolist()
```

```
X = pd.get_dummies(X, columns=cat_cols, drop_first=False)
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y, test_size=0.2, random_state=42)
```

```
# ■■ Train Gradient Boosting Regressor ██████████
```

```
model = GradientBoostingRegressor()
```

```
n_estimators=200,
```

```
learning_rate=0.05,
```

```
max depth=4,
```

```
random state=42
```

)

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
# Metrics
```

```
mae = mean_absolute_error(y_test, y_pred)
```

```
rmse = mean_squared_error(y_test, y_pred, squared=False)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print(f'MAE:  ${mae:.2f}')
```

```
print(f'RMSE: ${rmse:.2f}')
```

```
print(f'R²: {r2:.4f}')
```

[illegible]

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```
axes[0].scatter(y_test, y_pred, alpha=0.3, color='#3949AB')
```

```
axes[0].plot([y_test.min(), y_test.max()],
```

```
[y_test.min(), y_test.max()], 'r--', lw=2)
```

```
axes[0].set_xlabel('Actual LTV'); axes[0].set_ylabel('Predicted LTV')
```

```
axes[0].set_title(f'LTV: Predicted vs Actual (R2= $\{r2:.3f\}$ )')
```

```
residuals = y_test - y_pred
axes[1].hist(residuals, bins=50, color='#3949AB', alpha=0.7)
axes[1].set_xlabel('Residual ($)'); axes[1].set_ylabel('Count')
axes[1].set_title('LTV Residuals Distribution')
axes[1].axvline(0, color='red', ls='--')

plt.tight_layout()
plt.savefig('reports/figures/17_ltv_regression.png', dpi=150, bbox_inches='tight')
plt.show()

joblib.dump(model, 'models/ltv_regression.pkl')
joblib.dump(list(X.columns), 'models/ltv_feature_columns.pkl')
print('LTV regression model saved')
```

■ 3. Commit & Checklist

- Gradient Boosting Regressor trained for LTV
- MAE, RMSE, R^2 metrics printed
- Predicted vs actual scatter plot saved

■ Week3-Day2 Commit

```
git add .
git commit -m "Week3-Day2: LTV regression model – GBR trained, MAE/R2 metrics"
git push origin main
```

■ Estimated Time: 5 – 6 hours

— End of Week 3 Day 2 — Zaalima Development Confidential —